

I. REQUIREMENTS AND SPECIFICATIONS

Our SCADA and SDA systems were developed under many self-imposed design constraints, all of which were met during development. This system was publicly demonstrated at the 2026 Cal Poly Pomona Senior Project Symposium in Building 9, 401.

Hardware Constraints

1. The system should be able to maintain a water level within a user specified tolerance.
2. The system should be able to detect water level using an analog sensor.
3. The system should be able to actuate motors and valves to control the water level and water movement.

Manual Control Constraints

1. The system should have physical switches that allow hardware actuators to be enabled or disabled.
2. The system should have an emergency off switch to kill all actuator power without unpowering the SCADA system.
3. The system should have software override capabilities (software point holds) that allow the user to hold a variable (such as a sensor reading or output) at a certain specified level.

Internal Control Constraints

1. Each DCS should be able to control its actuators based on sensor input and user configured logical conditions.
2. All intermediate variables should be reported to the SCADA system for logging and display.

Communication Constraints

1. The SCADA system is connected to all active DCS systems via USB serial connection.
2. The SCADA system can be accessed over local WIFI for control and data retrieval.
3. The SCADA system can be accessed by SSH for headless control and CLI usage.

Software Constraints and Requirements

1. The SCADA software will feature both CLI and GUI user control that can be accessed locally or over the web (within local network).
2. The SCADA software will be self-deploying when ran, creating all folders and needed files at a user specified file location which can be moved.
3. The SCADA software will be multi-threaded such that user commands do not block server/supervisor operations.
4. The SCADA software will feature a **serial monitoring** thread to make serial connection management and detection non-blocking.
5. The SCADA software will feature a **FLASK back-end** thread to make user input and requests non-blocking.
6. The SCADA software will feature a **flashing** thread to make writing code to Arduino AVR boards non-blocking. This will use the Arduino IDE CLI.
7. The SCADA software will feature a feature complete code-block logic system with C++ code-generation. This

should feed into the **flashing** thread via an upload pipeline.

8. The SCADA software should feature a functionally complete CLI that can access server commands.
9. The SCADA software should feature a functionally complete GUI that can utilize server commands to display information to the user.